

Streaming & Structured Outputs

Deliver progress now and a validated value at completion

Source: <https://learn.microsoft.com/en-us/agent-framework/concepts/agents/running-agents?tabs=python> · +1 in notes

One run API, two return shapes

- **Non-streaming**

- `await agent.run(...)`
- Returns `AgentResponse`
- Read `.text`, `.value`, and typed contents

```
result = await agent.run("What is the weather like in Amsterdam?")
print(result.text)
```

- **Streaming**

- `agent.run(..., stream=True)`
- Returns `ResponseStream`
- Iterate `AgentResponseUpdate` values

```
async for update in agent.run("What is the weather like in Amsterdam?", stream=True):
    if update.text:
        print(update.text, end="", flush=True)
```

Three ways to consume `ResponseStream`

- **Iterate only** — Render each update as it arrives
- **Finalize only** — Skip iteration; finalizer consumes the stream
- **Iterate + finalize** — Render updates, then await `get_final_response()`

Updates carry typed contents

- **Text** — Convenient `update.text` for display
- **Function activity** — Calls and results can arrive as typed contents
- **Metadata** — Response IDs, roles, usage, and provider data may be present
- **Rule** — Ignore unknown types safely; do not flatten every event into text

```
# Access content from responses
response = await agent.run("Describe the image")
for message in response.messages:
    for content in message.contents:
        if content.type == "text":
            print(f"Text: {content.text}")
        elif content.type == "data":
            print(f>Data URI: {content.from_data}")
        elif content.type == "uri":
            print(f"External URI: {content.uri}")
```

Pydantic gives you a typed value

```
from pydantic import BaseModel

class TriageResult(BaseModel):
    route: str
    summary: str
    needs_work_item: bool

response = await agent.run(
    request,
    options={"response_format": TriageResult},
)
triage: TriageResult = response.value
```

JSON Schema works without a model class

```
schema = {  
    "type": "object",  
    "properties": {"route": {"type": "string"}},  
    "required": ["route"],  
}  
  
response = await agent.run(  
    request, options={"response_format": schema}  
)  
result = response.value  # parsed JSON
```

Example: TriageResult fits your docs assistant

- **Classify** — route: answer, clarify, or work_item
- **Summarize** — concise issue statement for your UI
- **Signal** — needs_work_item is a recommendation
- **Authorize** — your app still controls whether a write is offered or approved

Combine streaming and structure

```
stream = agent.run(
    request,
    stream=True,
    session=session,
    options={"response_format": TriageResult},
)

async for update in stream:
    if update.text:
        render(update.text)

final = await stream.get_final_response()
triage = final.value
```


D E M O

~5 min

Watch it stream, then get the typed TriageResult

Run the previous slide's exact combined pattern live: text tokens render as they arrive, then `stream.get_final_response().value` prints a validated `TriageResult` — the two-contract model made concrete on the Day 2 docs assistant.

Runbook: [demos/day3/module-2-demo-1-stream-then-triage.md](#)

Partial JSON is display data, not a value

- **During the stream**
 - JSON may be incomplete
 - Fields may not have arrived
 - Do not parse or act
- **After completion**
 - Await `get_final_response()`
 - Read validated `.value`
 - Apply business rules and approvals

Choose the response contract

Need	Pattern
Fastest simple code	Await AgentResponse
Progressive display	Iterate ResponseStream
Typed handoff	Await response and read .value
Progressive display + typed handoff	Iterate, then get_final_response().value
Dynamic contract	JSON Schema mapping

Takeaways

- ✓ You use one run API and choose AgentResponse or ResponseStream.
- ✓ You treat updates as typed contents, not only text.
- ✓ You put response_format inside options.
- ✓ You parse structured output only after the stream completes.
- ✓ You verify support with your exact Foundry client and model.